

# claude-windows / 902e27a6-e026-4b55-8c26-f2c47329cc44

## Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\902e27a6-e026-4b55-8c26-f2c47329cc44\subagents\workflows\wf_c287c071-8e9\agent-aa08899520aa6f0ed.jsonl`  
- SHA-256: `a85768fb88a7a7c8429a5808511e5819ae61014d07e96530e0d53aad2cf9a81a`  
- Source modified: `2026-07-07T14:49:25+00:00`  
- Imported at: `2026-07-08T16:00:01+00:00`  
- project: `wf_c287c071-8e9`  
- session_id: `902e27a6-e026-4b55-8c26-f2c47329cc44`
```

## Transcript

### 1. user (2026-07-07T14:45:11.299Z)

You are reviewing an uncommitted change in a git worktree at:  
C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval  
Branch: fix/worker-skip-revalidation-on-dispatch (based on origin/master).

THE BUG BEING FIXED: On the cloud-serving path, the control-plane (backend/orchestration.py) runs validators and ONLY dispatches to the Cloud Run worker on PASS. The worker (backend/worker/\_\_main\_\_.py cmd\_infer) then REDUNDANTLY re-ran validation with its OWN config (different min\_blur\_threshold), rejecting a clip the control-plane already passed. Live failure 2026-07-07: passed at min\_blur\_threshold=8.0 on the control-plane, worker rejected at its default 20.0 ? GPU dispatch succeeded but the CUJ failed with worker rc=2.

THE FIX (skip-on-dispatch): a new ComputeJobSpec.skip\_validation bool (default False) ? backend/compute/cloud\_run.py emits a "--skip-validation" worker CLI flag when set ? backend/worker/\_\_main\_\_.py cmd\_infer honors it (skips validator\_registry.run\_all\_with\_context, sets val\_results=[], val\_context={}) ? backend/orchestration.py::\_maybe\_dispatch\_remote sets skip\_validation=True on every dispatch. The direct "worker infer" CLI (no control-plane) still validates by default. New tests in tests/test\_worker.py and tests/test\_api\_cloud\_dispatch.py.

Inspect the ACTUAL code ? do NOT trust this summary. Run: cd "C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval" && git --no-pager diff and read the surrounding functions (not just the diff hunks). The full test suite already passes (1779 passed), ruff + mypy are clean, coverage 85.42%. So do NOT report style nits or things the gate already covers. Hunt for REAL correctness/rollout/consumer defects the tests would miss. Only report a finding you can tie to a concrete failure scenario in THIS repo's code.

LENS: COMPLETENESS OF THE WIRING. Is skip\_validation threaded through EVERY path that dispatches a

worker infer job? Search the whole repo (cd "C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval") for:

- every construction of ComputeJobSpec (git grep -n "ComputeJobSpec") ? does each dispatch site that should skip actually pass skip\_validation? Is there a dispatch site (besides orchestration \_maybe\_dispatch\_remote and worker \_dispatch\_remote) that was missed?
- every ComputeBackend implementation / anything that turns a ComputeJobSpec into a worker argv (git grep -rn "run\_infer\\|\\-\\-video-uri\\|def run\_job\\|infer"" backend). Is there a backend OTHER than cloud\_run.py that must also emit --skip-validation but doesn't?
- is the control-plane guaranteed to have validated before every \_maybe\_dispatch\_remote call, so unconditional skip\_validation=True is safe? Check all call sites.

Report only genuine gaps where a dispatch would NOT carry the flag when it should (or would carry it

when the caller had NOT validated).

## 2. assistant (2026-07-07T14:45:15.857Z)

I'll start by examining the actual change and its surrounding context.

## 3. user (2026-07-07T14:45:17.865Z)

```
diff --git a/backend/compute/base.py b/backend/compute/base.py
index 279606e..31af3fe 100644
--- a/backend/compute/base.py
+++ b/backend/compute/base.py
@@ -28,6 +28,13 @@ class ComputeJobSpec:
     segmenter: Optional[str] = None
     # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
    frozen/hashable.
     config_overrides: tuple[str, ...] = ()
+    # Skip the worker's redundant re-validation. The DISPATCHER (the cloud-serving control-
    plane) is
+    # the authoritative validation gate: it runs the SAME validators with ITS resolved config
    and only
+    # dispatches on PASS, so a worker re-run is (a) redundant, (b) a second full-video decode,
    and
+    # (c) outright WRONG when the worker's config resolves different thresholds ? it can REJECT
    a clip
+    # the gate already passed. Default False keeps the direct ``worker infer`` CLI (no control-
    plane)
+    # validating exactly as before.
+    skip_validation: bool = False
```

```
@dataclass(frozen=True)
diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
index 841785d..7247ace 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -196,6 +196,10 @@ class CloudRunCompute(ComputeBackend):
    ]
    if spec.segmenter:
        argv += ["--segmenter", spec.segmenter]
+    if spec.skip_validation:
+        # The control-plane already validated + only dispatches on PASS ? tell the worker
    not to
+        # re-decode and re-validate (redundant, and its config may resolve different
    thresholds).
+        argv.append("--skip-validation")
        for kv in (*spec.config_overrides, *self._container_overrides):
            argv += ["--set", kv]
```

```
diff --git a/backend/orchestration.py b/backend/orchestration.py
index 5e9f510..79b4fed 100644
--- a/backend/orchestration.py
+++ b/backend/orchestration.py
@@ -568,11 +568,18 @@ def _maybe_dispatch_remote(
    prefetch = getattr(wasb_cfg, "prefetch_batches", None)
    if prefetch:
        overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
+    # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
    authoritative
+    # gate ? it already ran the validators with its resolved config just above (only a PASS
    reaches
+    # here; a failure returned "failed" before dispatch). A worker re-run would decode the
    video a
+    # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
    thresholds and
```

```

+ # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
+ # then worker-rejected at its default 20.0 after a successful GPU dispatch).
spec = ComputeJobSpec(
    video_uri=req.video_path,
    out_uri=out_root,
    segmenter=seg_name,
    config_overrides=tuple(overrides),
+ skip_validation=True,
)
try:
    # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index 88c50f6..9b19215 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -186,6 +186,7 @@ def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
    out_uri=args.out_uri,
    segmenter=args.segmenter,
    config_overrides=tuple(args.set or ()),
+ skip_validation=bool(getattr(args, "skip_validation", False)),
)
try:
    result = compute.run_infer(spec)
@@ -259,10 +260,25 @@ def cmd_infer(args: argparse.Namespace) -> int:
    # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be byte-
identical).
    video_id = compute_video_id(local_video)

- # 3. VALIDATE (same registry as the main CLI).
- val_results, val_context = validator_registry.run_all_with_context(
-     local_video, config
- )
+ # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
+ # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate: it runs
the
+ # SAME validators with ITS resolved config and only dispatches on PASS. Re-validating here
is
+ # redundant, decodes the whole video a SECOND time, and is outright WRONG when the worker's
config
+ # resolves different thresholds than the control-plane's ? observed 2026-07-07: a 5312x2988
clip
+ # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0, then the
worker
+ # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the fact.
+ # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote) skips
it; the
+ # direct ``worker infer`` CLI has no control-plane, so it still validates by default.
+ if getattr(args, "skip_validation", False):
+     print(
+         "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
+         "gate and already validated this input."
+     )
+     val_results, val_context = [], {}
+ else:
+     val_results, val_context = validator_registry.run_all_with_context(
+         local_video, config
+     )
    failures = [r for r in val_results if not r.passed]
    db = Database(os.path.join(scratch, config.output.db_name))
    db.add_video(
@@ -724,6 +740,14 @@ def build_parser() -> argparse.ArgumentParser:
    metavar="k=v",
    help="config override, e.g. --set indexing.skip_ai_handoff=true",

```

```

    )
+   pi.add_argument(
+       "--skip-validation",
+       action="store_true",
+       help="skip the worker's re-validation: the dispatcher (control-plane) already ran the "
+       "validators with its resolved config and only dispatches on PASS, so re-validating here "
+       "is "
+       "redundant, decodes the video again, and can contradict the gate. Default-OFF: the "
direct "
+       "`worker infer` CLI still validates.",
+   )
+   pi.set_defaults(func=cmd_infer)

    pt = sub.add_parser(
diff --git a/tests/test_api_cloud_dispatch.py b/tests/test_api_cloud_dispatch.py
index 8382234..19c37d9 100644
--- a/tests/test_api_cloud_dispatch.py
+++ b/tests/test_api_cloud_dispatch.py
@@ -236,6 +236,22 @@ def test_cloud_dispatch_omits_unset_wasb_tuning(cloud_env):
    assert "indexing.wasb.prefetch_batches" not in joined

+def test_cloud_dispatch_skips_worker_revalidation(cloud_env):
+    """The control-plane is the AUTHORITATIVE validation gate: it validated + only dispatches
on PASS
+    (registry.run_all_with_context is stubbed to pass in cloud_env). Re-validating on the
worker is
+    redundant, decodes the video a second time, and ? because the control-plane's validation
config
+    isn't forwarded ? can REJECT a clip the gate already passed (observed 2026-07-07: passed at
+    validation.min_blur_threshold=8.0, then worker-rejected at its default 20.0). So the
dispatch MUST
+    carry --skip-validation, telling the worker to trust the gate. Regression guard for that
wiring."""
+    _db, _bucket, monkeypatch = cloud_env
+    fake = _FakeCloudClient()
+    _inject_fake(monkeypatch, fake)
+    m._execute_processing(
+        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
+    )
+    assert "--skip-validation" in fake.calls[0]
+
+def test_cloud_failure_marks_failed_no_rows(cloud_env):
+    db, _bucket, monkeypatch = cloud_env
+    # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-confirm).
diff --git a/tests/test_worker.py b/tests/test_worker.py
index 8fb5e16..72b2514 100644
--- a/tests/test_worker.py
+++ b/tests/test_worker.py
@@ -97,6 +97,14 @@ def test_parser_infer_accepts_set_and_uris():
    assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]

+def test_parser_infer_skip_validation_flag():
+    base = ["infer", "--video-uri", "s3://b/v.mp4", "--out-uri", "s3://b"]
+    # Default-OFF: the direct `worker infer` CLI still validates.
+    assert wmain.build_parser().parse_args(base).skip_validation is False
+    # The dispatcher passes it to disable the redundant worker re-validation.
+    assert wmain.build_parser().parse_args(base + ["--skip-validation"]).skip_validation is
True
+
+
+    # ----- cmd_infer (local, mocked)

```

```

@@ -407,6 +415,43 @@ def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
    assert marker["returncode"] == 2 and marker["status"] == "failed"

+def test_cmd_infer_skip_validation_bypasses_validators(tmp_path, monkeypatch):
+    """--skip-validation (set by the dispatcher, the authoritative gate) must NOT run the
worker's
+    validators ? no second decode, and no chance of a threshold mismatch REJECTING a clip the
+    control-plane already passed (the live 2026-07-07 CUJ failure). Prove it by making
+    run_all_with_context RAISE: a skipped run never touches it, so the job completes rc 0 with
a
+    status="success" report. Contrast test_cmd_infer_validation_failure_returns_2 (no flag ? rc
2)."""
+    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidSkip")
+
+    def _boom(*a, **k): # the validators must never be reached on the skip path
        raise AssertionError("run_all_with_context must not run under --skip-validation")
+
+    monkeypatch.setattr(wmain.validator_registry, "run_all_with_context", _boom)
+    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
+    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
+    video = tmp_path / "in.mp4"
+    video.write_bytes(b"FAKE")
+    out = tmp_path / "bucket"
+
+    rc = wmain.main(
+        [
+            "infer",
+            "--video-uri",
+            str(video),
+            "--out-uri",
+            str(out),
+            "--scratch",
+            str(tmp_path / "s"),
+            "--segmenter",
+            "wasb_hybrid",
+            "--skip-validation",
+        ]
+    )
+    assert rc == 0 # a clip that WOULD fail re-validation still succeeds ? the gate is trusted
+    report = json.loads((out / "outputs" / "vidSkip" / "report.json").read_text())
+    assert report["status"] == "success" and report["segment_count"] == 2
+
+
# --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle ---

```

#### 4. user (2026-07-07T14:45:18.990Z)

```

backend/orchestration.py:577: spec = ComputeJobSpec(
backend/worker/__main__.py:184: spec = ComputeJobSpec(
tests/test_compute_backend.py:184: ComputeJobSpec(
tests/test_compute_backend.py:209: ComputeJobSpec(
tests/test_compute_backend.py:237: ComputeJobSpec(video_uri="gs://b/v.mp4",
out_uri=str(tmp_path / "bk"))
tests/test_compute_backend.py:240: ComputeJobSpec(video_uri="gs://b/v.mp4",
out_uri=str(tmp_path / "bk"))
tests/test_compute_backend.py:308: ComputeJobSpec(video_uri="gs://b/v.mp4",
out_uri="gs://b")
tests/test_compute_backend.py:335: ComputeJobSpec(video_uri="gs://b/v.mp4",
out_uri=str(tmp_path / "bucket"))
tests/test_compute_backend.py:357: ComputeJobSpec(video_uri="gs://b/v.mp4",
out_uri=str(tmp_path / "bucket"))
tests/test_compute_backend.py:394: ComputeJobSpec(video_uri="gs://b/v.mp4",
out_uri="gs://b")
tests/test_compute_backend.py:780: ComputeJobSpec(video_uri="gs://b/v.mp4",

```

```
out_uri=str(tmp_path / "bk")),
```

## 5. assistant (2026-07-07T14:45:24.474Z)

Let me look at the compute backend implementations and all dispatch call sites.

## 6. user (2026-07-07T14:45:27.187Z)

```
__init__.py
__pycache__
base.py
cloud_run.py
---BASE---
"""The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.

COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
(``backend/storage/``): a ``ComputeBackend`` decides **where** the pure DETECT?WINDOW?STITCH
core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
(cloud-serving) ? without the core ever branching on the profile (D1).

The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
non-local backend. Every other target (``""`` / ``local_gpu`` / ``cpu_mock``) keeps the worker's
existing in-process path **byte-identical** ? the factory returns ``None`` (= "run inline").
"""

from __future__ import annotations

from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from typing import Callable, Optional

@dataclass(frozen=True)
class ComputeJobSpec:
    """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will (re)compute
    from the SAME input bytes, so the output always lands at ``<out_uri>/outputs/<video_id>/`` ?
    deterministic regardless of where the job ran."""

    video_uri: str # input (gs://? for cloud; a path/local:// for local)
    out_uri: str # output ROOT (gs://? bucket for cloud)
    segmenter: Optional[str] = None
    # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
    frozen/hashable.
    config_overrides: tuple[str, ...] = ()
    # Skip the worker's redundant re-validation. The DISPATCHER (the cloud-serving control-
    plane) is
    # the authoritative validation gate: it runs the SAME validators with ITS resolved config
    and only
    # dispatches on PASS, so a worker re-run is (a) redundant, (b) a second full-video decode,
    and
    # (c) outright WRONG when the worker's config resolves different thresholds ? it can REJECT
    a clip
    # the gate already passed. Default False keeps the direct ``worker infer`` CLI (no control-
    plane)
    # validating exactly as before.
    skip_validation: bool = False

@dataclass(frozen=True)
class ComputeResult:
    """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit code
    (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch timeout or
    an
    empty/unreadable completion marker** (the cloud path only). ``video_id`` is known once the
    job's
```

```

done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json`` landed."""

returncode: int
outputs_uri: str = ""
video_id: str = ""
report: dict = field(default_factory=dict)
# The backend's execution identity for PROVENANCE (e.g. the Cloud Run execution name) ?
surfaced
# on the job result so "the cloud path actually ran" is independently verifiable (gcloud run
jobs
# executions describe <execution>). Empty for the in-process path / backends without one.
execution: str = ""

class ComputeBackend(ABC):
    """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns only
    once
    its outputs are durably available (bucket-poll for cloud), so a caller treats local and
    cloud
    execution identically."""

    @abstractmethod
    def run_infer(
        self,
        spec: ComputeJobSpec,
        *,
        on_wait: "Callable[[float], None] | None" = None,
    ) -> ComputeResult:
        """Run ``spec`` to completion and return its ``ComputeResult``.

        ``on_wait(elapsed_sec)`` (optional) is invoked periodically while the backend is
        still WAITING on the remote work ? the caller's live-progress heartbeat (#482), so a
        long GPU run doesn't look frozen. Implementations without a wait loop may ignore it."""

```

## 7. user (2026-07-07T14:45:28.459Z)

```

backend/compute/base.py:57:class ComputeBackend(ABC):
backend/compute/base.py:58:     """Backend-agnostic compute. ``run_infer`` dispatches ONE
inference job and returns only once
backend/compute/base.py:63:     def run_infer(
backend/compute/cloud_run.py:61:         def run_job(
backend/compute/cloud_run.py:84:         def run_job(
backend/compute/cloud_run.py:149:class CloudRunCompute(ComputeBackend):
backend/compute/cloud_run.py:171:         # None ? a fresh uuid4 per run_infer call (prod);
injected ? deterministic (tests).
backend/compute/cloud_run.py:179:         def run_infer(
backend/config/loader.py:205:             # active profile's compute_target (it stays
meaningful for the M2+ ComputeBackend).
backend/orchestration.py:589:         result = compute.run_infer(
backend/worker/__main__.py:192:         result = compute.run_infer(spec)

```

## 8. user (2026-07-07T14:45:37.783Z)

```

"""Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.

COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and ``None``
otherwise ? ``None`` means "run the worker's existing in-process path" (today's behaviour, byte
identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` / ``cpu_mock``
nothing
changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
"""

from __future__ import annotations

```

```

import dataclasses
import os
from typing import Any, Optional

from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy

__all__ = [
    "ComputeBackend",
    "ComputeJobSpec",
    "ComputeResult",
    "get_compute_backend",
]

def _storage_dict(config: Any) -> dict:
    storage = getattr(config, "storage", None)
    if storage is None:
        return {}
    if hasattr(storage, "model_dump"):
        return storage.model_dump() # type: ignore[no-any-return]
    return dict(storage) if isinstance(storage, dict) else {}

def get_compute_backend(
    config: Any,
    *,
    run_client: Optional[Any] = None,
    policy: ExecutionPolicy = DEFAULT_POLICY,
    token: Optional[str] = None,
    target_override: Optional[str] = None,
) -> Optional[ComputeBackend]:
    """Return the compute backend for ``config.deployment.compute_target``, or ``None`` to run
    in-process (the default-OFF path).

    Only ``compute_target == "cloud_run"`` returns a non-None backend (a ``CloudRunCompute``).
    Its
    Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
    ``CLOUD_RUN_REGION``
    / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is injectable
    (tests
    pass a fake; production defaults to the real ``google-cloud-run`` client, owner-verified on
    M7).

    ``target_override`` lets a single request force a target (the SPA compute-picker maps a
    per-request ``compute="cloud"`` to ``target_override="cloud_run"``) without mutating the
    server's configured default. ``None`` falls back to ``config.deployment.compute_target``.
    """
    target = (
        target_override
        if target_override is not None
        else getattr(getattr(config, "deployment", None), "compute_target", "") or ""
    )
    if target != "cloud_run":
        return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)

    # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually selected.
    from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient

    # Derive the poll budget from config UNLESS the caller injected an explicit policy ?
    # deployment.remote_poll_max_wait_sec must outlast the worker Job's --task-timeout so a
    # slow-but-healthy paid GPU run is never abandoned mid-flight (CP-rebuild $4.3; the old
    # 1800s default undercut real 1080p60 jobs). Identity check on DEFAULT_POLICY keeps an
    # explicitly-passed policy (tests, callers with bespoke budgets) untouched.
    if policy is DEFAULT_POLICY:

```



```

max_wait = getattr(
    getattr(config, "deployment", None), "remote_poll_max_wait_sec", None
)
if max_wait:
    policy = dataclasses.replace(DEFAULT_POLICY, max_wait_sec=float(max_wait))

job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
project = os.environ.get("GOOGLE_CLOUD_PROJECT")
client = run_client if run_client is not None else GoogleCloudRunJobClient()
return CloudRunCompute(
    run_client=client,
    job_name=job_name,
    region=region,
    project=project,
    storage_config=_storage_dict(config),
    policy=policy,
    token=token,
)
)
===FACTORY/other backends===
backend/api/job_worker.py:223:         compute_backend=(result or {}).get("compute_backend"),
backend/compute/__init__.py:3:COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is
the seam: it returns a
backend/compute/__init__.py:4:``ComputeBackend`` ONLY when a non-local
``deployment.compute_target`` is selected, and ``None``
backend/compute/__init__.py:16:from backend.compute.base import ComputeBackend, ComputeJobSpec,
ComputeResult
backend/compute/__init__.py:20:     "ComputeBackend",
backend/compute/__init__.py:23:     "get_compute_backend",
backend/compute/__init__.py:36:def get_compute_backend(
backend/compute/__init__.py:43:) -> Optional[ComputeBackend]:
backend/compute/base.py:1:``The ``ComputeBackend`` ABC ? one interface for *where* the rally
core runs.
backend/compute/base.py:4:(``backend/storage/``): a ``ComputeBackend`` decides **where** the
pure DETECT?WINDOW?STITCH
backend/compute/base.py:57:class ComputeBackend(ABC):
backend/compute/base.py:58:     """Backend-agnostic compute. ``run_infer`` dispatches ONE
inference job and returns only once
backend/compute/base.py:63:     def run_infer(
backend/compute/cloud_run.py:35:from backend.compute.base import ComputeBackend, ComputeJobSpec,
ComputeResult
backend/compute/cloud_run.py:61:     def run_job(
backend/compute/cloud_run.py:84:     def run_job(
backend/compute/cloud_run.py:98:         "to resolve the job path; set it on the control
plane (see get_compute_backend)."
```

```

backend/compute/cloud_run.py:149:class CloudRunCompute(ComputeBackend):
backend/compute/cloud_run.py:179:     def run_infer(
backend/config/loader.py:205:         # active profile's compute_target (it stays
meaningful for the M2+ ComputeBackend).
backend/config/models.py:470:     # (recorded for observability; consumed by the ComputeBackend
seam from M2+).
backend/infrastructure/database.py:387:         ("compute_backend", "TEXT"), # local_gpu |
cloud_run | ...
backend/infrastructure/database_jobs.py:353:         compute_backend: Optional[str] = None,
backend/infrastructure/database_jobs.py:385:         "UPDATE jobs SET owner_id=?,
compute_backend=?, gpu_type=?, gpu_active_seconds=?, "
backend/infrastructure/database_jobs.py:390:         compute_backend,
backend/infrastructure/database_jobs.py:418:         "SELECT id, video_id, owner_id,
compute_backend, gpu_type, gpu_active_seconds, "
backend/orchestration.py:409:     (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for
every non-``cloud_run`` target, so
backend/orchestration.py:411:     from backend.compute import ComputeJobSpec, get_compute_backend
backend/orchestration.py:457:         compute = get_compute_backend(cfg,
target_override="cloud_run")
backend/orchestration.py:459:         compute = get_compute_backend(

```

```

backend/orchestration.py:589:         result = compute.run_infer(
backend/worker/__main__.py:30:from backend.compute import ComputeJobSpec, get_compute_backend
backend/worker/__main__.py:175:     """M6: hand ONE infer job to a remote ComputeBackend (Cloud
Run) and return its rc once the
backend/worker/__main__.py:192:         result = compute.run_infer(spec)
backend/worker/__main__.py:241:     # DEFAULT-OFF: get_compute_backend returns None for
"""/local_gpu/cpu_mock ? fall through to the
backend/worker/__main__.py:244:     compute = get_compute_backend(config)

```

## 9. user (2026-07-07T14:45:38.726Z)

```

    try:
        notify("staging upload to bucket")
        # Thread the active storage settings so backend-specific config survives (S3
endpoint_url/
        # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh default
client.
        storage.put(
            local_src,
            staged_uri,
            config=(
                storage_cfg.model_dump() if storage_cfg is not None else None
            ),
        )
    except Exception as e: # noqa: BLE001 ? a staging failure is a clean job failure, not a
500
        logger.error(
            "[API] cloud-serving: failed to stage %s -> %s: %s",
            local_src,
            staged_uri,
            e,
            exc_info=True,
        )
        return _failed(
            f"cloud-serving: failed to stage the uploaded file to {input_root}: {e}",
            False,
        )
    # Repoint BOTH the dispatch and the DB video record at the staged bucket object: the L4
job
    # DETECTs from it, and a later /api/compile re-fetches the same source (the local upload
is
    # gone once this instance recycles). add_video is an UPSERT ? it only updates filepath.
    req.video_path = staged_uri
    input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
    db.add_video(
        video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
    )
    out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
    if not out_root:
        return _failed(
            "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-bucket) "
            "? that's where the Cloud Run job writes the result.",
            False,
        )
    )
    notify("dispatching (cloud_run)")
    logger.info(
        "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
        req.video_path,
        out_root,
    )
    # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection: with
    # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key needed);
without
    # forwarding the cloud job would default to the handoff and yield 0 rallies when it has no

```

```

key.
# (player_pool / max_frames are still dropped on the cloud path ? a separate documented
follow-up.)
overrides = []
idx_cfg = getattr(cfg, "indexing", None)
sk = getattr(idx_cfg, "skip_ai_handoff", None)
if sk is not None:
    overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
# Forward the resolved WASB serving knobs so the PRESET governs the worker (#506/#507). The
# worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving preset itself
?
# without forwarding, its decode_backend + GPU-feed tuning silently fall back to the model
# defaults (cpu, batch 8) regardless of what the control-plane resolved. decode_backend is
# cuda-only (validator-enforced) and requires a worker image that understands --decode-
backend
# (? #507); batch_size/prefetch_batches are forwarded only when set (None = the parity
default).
wasb_cfg = getattr(idx_cfg, "wasb", None)
if wasb_cfg is not None:
    decode_backend = getattr(wasb_cfg, "decode_backend", None)
    if decode_backend:
        overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
    batch_size = getattr(wasb_cfg, "batch_size", None)
    if batch_size:
        overrides.append(f"indexing.wasb.batch_size={batch_size}")
    prefetch = getattr(wasb_cfg, "prefetch_batches", None)
    if prefetch:
        overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
# Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
# gate ? it already ran the validators with its resolved config just above (only a PASS
reaches
# here; a failure returned "failed" before dispatch). A worker re-run would decode the video
a
# second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
# REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
# then worker-rejected at its default 20.0 after a successful GPU dispatch).
spec = ComputeJobSpec(
    video_uri=req.video_path,
    out_uri=out_root,
    segmenter=seg_name,
    config_overrides=tuple(overrides),
    skip_validation=True,
)
try:
    # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
    # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
    # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ? which
    # also gives the SPA a stall signal to time out on, instead of a blind wall clock.
    result = compute.run_infer(
        spec,
        on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
    )
except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job failure, not
a 500
    logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
    return _failed(f"cloud dispatch error: {e}", True)
if result.returncode != 0:
    return _failed(f"cloud job failed (returncode {result.returncode}).", True)
if result.video_id and result.video_id != video_id:
    # The worker re-hashes the bucket bytes; a divergence means the input changed between
this
    # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's

```

```

diagnosable.
    logger.warning(
        "[API] cloud video_id %s != local %s ? input bytes differ between the API hash "
        "and the worker hash; ingesting under the local id.",
        result.video_id,
        video_id,
    )

    notify("ingesting (cloud_run)")
    storage_dict = (
        storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy can't
narrow
        if hasattr(storage_cfg, "model_dump")
        else {}
    )
    try:
        _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
    except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not 500 or
leak rows
        logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
        return _failed(
            f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
            True,
        )

    # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then drops the
old ones.
    segments = [
        s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
    ]
    _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
    # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution` is the
real
    # Cloud Run execution name (cross-check: gcloud run jobs executions describe <execution>).
    provenance = {
        "path": "cloud_run",
        "requested": req.compute,
        "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
        "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
        "execution": result.execution,
        "outputs_uri": result.outputs_uri,
    }
    logger.info(
        "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s video_id=%s",
        result.execution,

```

## 10. user (2026-07-07T14:45:46.400Z)

```

    notify,
) -> Optional[Tuple[Dict[str, Any], bool]]:
    """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
`deployment.compute_target
    == "cloud_run"` , run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally intervals
into
    THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.

    Returns `(response_dict, ran)` when the cloud path is taken ? `ran` is whether the cloud
job
    actually EXECUTED (`False` for a pre-dispatch config rejection, so the observability
report
    doesn't claim a fake segmentation run) ? or `None` to fall through to the in-process
segmenter
    (**DEFAULT-OFF**: `get_compute_backend` returns `None` for every non-`cloud_run`
target, so
    the in-process path stays byte-identical)."""

```

```

from backend.compute import ComputeJobSpec, get_compute_backend

def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
    # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows (id >
play_wm);
    # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
    db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
    # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run" only when
the
    # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker rc /
ingest
    # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target` records
the
    # intended backend, `requested` the picker choice, `dispatched` whether the remote job
ran.
    return (
        {
            "video_id": video_id,
            "status": "failed",
            "message": msg,
            "results": [r.model_dump() for r in val_results],
            "play_segments": [],
            "compute": {
                "path": "cloud_run" if ran else "not_run",
                "requested": req.compute,
                "target": "cloud_run",
                "dispatched": ran,
            },
        },
        ran,
    )

# SPA compute-picker (item 3): a per-request choice overrides the server's configured
default.
# "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch (guarded by
# _cloud_available so we fail clearly rather than silently running local); None / unknown ?
default.
choice = (req.compute or "").strip().lower()
if choice and choice not in ("local", "cloud"):
    logger.warning(
        "[API] ignoring unknown compute=%r (expected 'local'|'cloud'); using server
default.",
        req.compute,
    )
    choice = ""
if choice == "local":
    return None # explicit "run on my machine" ? in-process regardless of the server's
target
if choice == "cloud":
    if not _cloud_available(cfg):
        return _failed(
            "cloud-serving was requested but this server isn't configured for it "
            "(no Cloud Run target). Pick 'run on my machine', or configure cloud serving.",
            False,
        )
    compute = get_compute_backend(cfg, target_override="cloud_run")
else:
    compute = get_compute_backend(
        cfg
    ) # server default (default-OFF unless cloud_run)
if compute is None:
    return None # default-OFF ? run the in-process segmenter below (unchanged)

storage_cfg = getattr(cfg, "storage", None)

```

```

# The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local to this
box.
try:
    input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
except ValueError as e:
    return _failed(
        f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
    )
if input_ref.backend == "local":
    # A browser upload lands on THIS box's local disk; the Cloud Run job (a different
machine)
    # can't read it. Stage it UP to the input bucket first, then dispatch from there (#461).
Only
    # possible when a CLOUD input_root is configured; without one we genuinely can't stage ?
fail.
    # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged URI below
is
    # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
    # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left at its
    # default, while a bare/local-looking root can't stage no matter what input_backend says
    # (resolve_input_uri only consults input_backend when input_root is EMPTY, and an empty
root
    # gives us nothing to build the staged URI under).
    input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
    try:
        root_backend = (
            storage.parse_uri(input_root).backend if input_root else "local"
        )
    except ValueError: # unsupported scheme in input_root ? nowhere real to stage to
        root_backend = "local"
    if root_backend == "local":
        return _failed(
            "cloud-serving needs a cloud input (e.g. gs://?), or a cloud storage.input_root
to "
            "stage local uploads into; a path local to this machine can't be read by the
Cloud "
            "Run job.",
            False,
        )
    local_src = input_ref.key # resolved local path of the upload
    staged_uri = f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
    try:

```

## 11. user (2026-07-07T14:45:55.136Z)

```

`?video=<md5>`` lands on grounded state after any restart/redeploy wipe. True ? the DB
has the video's rows now (re-query). Non-md5 ids return False without touching the
bucket; every failure degrades to False (the read just reports what the DB has)."""
vid = (video_id or "").strip().lower()
if not _MD5_HEX_RE.fullmatch(vid):
    return False
if not probe_process_cache(db, cfg, vid):
    return False
return cached_process_result(db, cfg, vid, "") is not None

def _maybe_dispatch_remote(
    cfg,
    db,
    req: "ProcessRequest",
    video_id: str,
    seg_name: str,
    val_results,
    play_wm: int,

```

```

    cand_wm: int,
    notify,
===CALLERS of _maybe_dispatch_remote===
backend/orchestration.py:4:Cloud Run dispatch + ingest path (`_maybe_dispatch_remote` /
`_ingest_remote_segments`),
backend/orchestration.py:391:def _maybe_dispatch_remote(
backend/orchestration.py:397:     val_results,
backend/orchestration.py:426:         "results": [r.model_dump() for r in val_results],
backend/orchestration.py:529:         video_id, staged_uri, "processed", [r.model_dump() for
r in val_results]
backend/orchestration.py:651:         "results": [r.model_dump() for r in val_results],
backend/orchestration.py:724:     val_results: List[Any] = []
backend/orchestration.py:728:     # _maybe_dispatch_remote when it dispatches. None ? nothing ran
(validation/config fail).
backend/orchestration.py:754:         val_results, val_context =
registry.run_all_with_context(local_video, config)
backend/orchestration.py:755:         failures = [r for r in val_results if not r.passed]
backend/orchestration.py:764:             [r.model_dump() for r in val_results],
backend/orchestration.py:770:             "results": [r.model_dump() for r in val_results],
backend/orchestration.py:777:             video_id, req.video_path, "processed", [r.model_dump()
for r in val_results]
backend/orchestration.py:789:             "results": [r.model_dump() for r in val_results],
backend/orchestration.py:805:     remote = _maybe_dispatch_remote(
backend/orchestration.py:806:         config, db, req, video_id, seg_name, val_results,
play_wm, cand_wm, notify
backend/orchestration.py:838:         "results": [r.model_dump() for r in val_results],
backend/orchestration.py:851:         "results": [r.model_dump() for r in val_results],
backend/orchestration.py:890:         val_results=val_results,

```

## 12. user (2026-07-07T14:46:01.535Z)

```

was_reingest = existing_video is not None

# typed AppConfig: attribute access for the default segmenter (with safe fallback)
default_seg = getattr(
    getattr(config, "indexing", None), "default_segmenter", "motion"
)
seg_name = req.segmenter or default_seg
segmenter_actual = None
segmentation_ran = False
val_results: List[Any] = []
val_context: Dict[str, Any] = {}
# Compute-path PROVENANCE (the validation method): which backend actually ran the DETECT ?
# "in_process" once the local segmenter is reached, set on the response as "cloud_run" by
# _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config fail).
compute_actual: Optional[str] = None
response: Optional[Dict[str, Any]] = None
try:
    if (
        existing_video
        and existing_video.get("status") == "processed"
        and not req.force
    ):
        cached_segments = db.query_play_segments(video_id, {})
        if cached_segments:
            logger.info(
                "[API] Reusing cached segmentation for video_id=%s; force=true bypasses
cache.",
                video_id,
            )
        response = _cached_process_payload(
            video_id,
            existing_video.get("validation_results", []),
            cached_segments,
            getattr(req, "compute", None),

```

```

    )
    return response

# 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
notify("validating")
logger.info(f"Running validations for {req.video_path}...")
val_results, val_context = registry.run_all_with_context(local_video, config)
failures = [r for r in val_results if not r.passed]

if failures:
    # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
    # status; it no longer destroys the video's prior good segments.
    db.add_video(
        video_id,
        req.video_path,
        "failed",
        [r.model_dump() for r in val_results],
    )
    response = {
        "video_id": video_id,
        "status": "failed",
        "message": _validation_failure_message(failures),
        "results": [r.model_dump() for r in val_results],
    }
    return response

# 2. Run segmenter & indexer
logger.info("[API] Video passed checks. Segmenting and indexing...")
db.add_video(
    video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
)

segmenter_cls = segmenter_registry.get(seg_name)
if not segmenter_cls:
    # Submit-time validation already 400s unknown names; this is the defensive
    # in-worker path (e.g. config default pointing at an unregistered segmenter).
    available = list(segmenter_registry.list_available().keys())
    response = {
        "video_id": video_id,
        "status": "failed",
        "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
        "results": [r.model_dump() for r in val_results],
        "play_segments": [],
    }
    return response

logger.info(f"[API] Using segmenter: {seg_name}")
notify(f"segmenting ({seg_name})")
segmenter_actual = seg_name
# Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old only
# if the run yields segments; otherwise drop the new partial rows and keep the old.
play_wm, cand_wm = db.segment_watermarks(video_id)
# COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run, run the
heavy
# DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
# DEFAULT-OFF: returns None for every non-cloud target ? the in-process segmenter below
runs
# byte-identically. (player_pool/max_frames aren't threaded to the worker `infer` CLI
yet, so
# they're dropped on the cloud path in this slice ? a documented follow-up.)
remote = _maybe_dispatch_remote(
    config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify

```



```

    )
    if remote is not None:
        response, segmentation_ran = (
            remote # `ran` = did the cloud job actually execute (report)
        )
        return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
    # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
    compute_actual = "in_process"
    segmenter = segmenter_cls(db, config)
    try:
        # Serialize the CPU/GPU-heavy in-process detection (see _LOCAL_COMPUTE_LANE). A
remote
        # dispatch never reaches here (it returned above), so remote polls still overlap
freely.
        with _LOCAL_COMPUTE_LANE:
            result = segmenter.process_video(
                video_id, local_video, req.player_pool, req.max_frames
            )
    except Exception:
        # A raising segmenter must not leave half-written rows: restore the pre-run
        # state (same destroy-after-confirm contract as the Failure branch), then let
        # the job worker record the failure.
        db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
        raise
    segmentation_ran = True

    if isinstance(result, Failure):
        logger.error(f"[API] Segmenter failed: {result.message}")
        db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
        response = {
            "video_id": video_id,
            "status": "failed",
            "message": f"Segmenter '{seg_name}' failed: {result.message}",
            "results": [r.model_dump() for r in val_results],
            "play_segments": [],
        }
    }
    return response

    segments = result.value if hasattr(result, "value") else result
    notify("finalizing")
    _finalize_reingest(db, video_id, segments, play_wm, cand_wm)

    response = {
        "video_id": video_id,
        "status": "success",
        "message": f"Successfully indexed {len(segments)} play segments using '{seg_name}'
segmenter.", # type: ignore[arg-type] # hasattr-unwrap above; Success arm has .value
        "results": [r.model_dump() for r in val_results],
        "play_segments": segments,
    }
    return response
finally:
    # Compute-path PROVENANCE stamp (the validation method): the cloud branch already set
    # response["compute"] (path=cloud_run + the Cloud Run execution); for every other path
stamp
    # the actual path here so the job result PROVES which backend ran (in_process / not_run)
and
    # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is then
visible.
    if isinstance(response, dict):
        response.setdefault(
            "compute",
            {

```

```

        "path": compute_actual or "not_run",
        "requested": getattr(req, "compute", None),
    },
)
prov = response.get("compute", {}) if isinstance(response, dict) else {}
logger.info(
    "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
    video_id,
    getattr(req, "compute", None),
    prov.get("path"),
    prov.get("dispatched"),
    (response or {}).get("status") if isinstance(response, dict) else None,
)
# Always emit the per-video observability report (next to the video, or to
# report.output_dir). Never raises. Surface the paths in the response. (The compute-path
# proof lives on the job result + the [COMPUTE] log, not the report ? obsreport's
record_run
# doesn't surface arbitrary run signals.)
paths = _main.emit_indexer_report(
    db=db,
    video_id=video_id,
    # Resolved LOCAL path, not the raw URI: obsreport writes the report next to the
    # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://, local://)
    # collapses to a literal scheme dir at the repo root (see #397). The source URI is
    # still preserved in DB provenance via db.add_video.
    video_path=local_video,
    config=_report_config_for_input(config, req.video_path),
    val_results=val_results,
    val_context=val_context,
    segmenter_requested=req.segmenter,
    segmenter_actual=segmenter_actual,
    was_reingest=was_reingest,
    segmentation_ran=segmentation_ran,
)
if paths and isinstance(response, dict):
    response["reports"] = paths
storage.cleanup_acquired_local_input(
    req.video_path, local_video, getattr(config, "storage", None)
)

```

### 13. user (2026-07-07T14:46:12.793Z)

```

except StartupCheckError:
    return False # already logged at ERROR by enforce_startup_checks

def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
    """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc once the
    outputs are durably in the bucket (the backend bucket-polls the done-marker). The dispatched
    container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never re-
    dispatch.

    A remote job that never writes a completion marker (a failed/hung container) makes the
    bucket
    poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4 (remote
    dispatch
    timeout/failure, distinct from the local 2/3) rather than letting a raw traceback escape."""
    from backend.infrastructure.execution_policy import PolicyTimeout

    spec = ComputeJobSpec(
        video_uri=args.video_uri,
        out_uri=args.out_uri,
        segmenter=args.segmenter,
        config_overrides=tuple(args.set or ()),
        skip_validation=bool(getattr(args, "skip_validation", False)),
    )

```

```

try:
    result = compute.run_infer(spec)
except PolicyTimeout as e:
    logger.warning(
        "[worker] remote compute did not complete (no marker within budget): %s", e
    )
    return 4
logger.info(
    "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
    result.video_id,
    result.returncode,
    result.outputs_uri,
)
return result.returncode

def _write_done_marker(
    done_uri: str,
    video_id: str,
    returncode: int,
    segment_count: int,
    status: str,
    storage_cfg: dict,
    scratch: str,
) -> None:
    """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer) for a
remote
dispatcher's bucket-poll. Best-effort ? never raises into the worker's success return."""
    try:
        marker = {
            "video_id": video_id,
            "returncode": returncode,
            "segment_count": segment_count,
            "status": status,
        }
        local = os.path.join(scratch, "_done.json")
        with open(local, "w", encoding="utf-8") as f:
            json.dump(marker, f)
        storage.put(local, done_uri, config=storage_cfg)
        logger.info("[worker] wrote completion marker -> %s", done_uri)
    except Exception as e: # never fail a good run because the marker push hiccuped
        logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)

def cmd_infer(args: argparse.Namespace) -> int:
    config = load_config(args.config) if args.config else load_config()
    _apply_overrides(config, args.set)
    if not _run_startup_checks(config):
        return 2

    # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-process.
    # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall through to
the
    # unchanged inline path below. The dispatched container runs THIS cmd_infer inline (the
    # dispatcher forces compute_target=local_gpu), so it never recursively re-dispatches.
    compute = get_compute_backend(config)
    if compute is not None:
        return _dispatch_remote(compute, args)

    storage_cfg = config.storage.model_dump()

    scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
    os.makedirs(scratch, exist_ok=True)
    config.output.output_dir = scratch

```

```

# 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download to
scratch.
local_video = storage.fetch(
    args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
)
print(f"[worker] pulled {args.video_uri} -> {local_video}")

# 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be byte-
identical).
video_id = compute_video_id(local_video)

# 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
# On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate: it runs
the
# SAME validators with ITS resolved config and only dispatches on PASS. Re-validating here
is
# redundant, decodes the whole video a SECOND time, and is outright WRONG when the worker's
config
# resolves different thresholds than the control-plane's ? observed 2026-07-07: a 5312x2988
clip
# (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0, then the
worker
# REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the fact.
# --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote) skips
it; the
# direct ``worker infer`` CLI has no control-plane, so it still validates by default.
if getattr(args, "skip_validation", False):
    print(
        "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
        "gate and already validated this input."
    )
    val_results, val_context = [], {}
else:
    val_results, val_context = validator_registry.run_all_with_context(
        local_video, config
    )
failures = [r for r in val_results if not r.passed]
db = Database(os.path.join(scratch, config.output.db_name))
db.add_video(
    video_id,
    local_video,
    "failed" if failures else "processed",
    [r.model_dump() for r in val_results],
)

def _fail(rc: int) -> int:
    # A worker-side FAILURE (validation, unknown segmenter, or a detector timeout/abort the
    # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success". Write +
push
    # a status="failed" report.json/intervals.csv (0 segments) so the control plane's bucket
    # truth (submit-cache probe, restart-recovery) and the alpha-user result surface see an
    # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup must never
    # mask the original failure. THEN the M6 done-marker (real rc + status) so a remote
    # dispatcher's bucket-poll terminates FAST instead of hanging until its budget expires.
    try:
        _serialize_and_push_outputs(
            scratch,
            args.out_uri,
            video_id,
            [],
            "failed",
            storage_cfg,
            str(getattr(args, "video_uri", "") or ""),
        )

```

```

except Exception as e: # never mask the real failure on an artifact-push hiccup
    logger.warning("[worker] could not push failure artifacts: %s", e)
# DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
if getattr(args, "done_uri", None):
    _write_done_marker(
        args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
    )
return rc

segments: List[dict] = []
segmenter_actual = None
segmentation_ran = False
status = "failed"
try:
    if failures:
        print(
            "[worker] validation FAILED: "
            + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
        )
        return _fail(2)
    seg_name = args.segmenter or config.indexing.default_segmenter
    segmenter_cls = segmenter_registry.get(seg_name)
    if not segmenter_cls:
        print(
            f"[worker] unknown segmenter: {seg_name!r} "
            f"(have {list(segmenter_registry.list_available())})"
        )
        return _fail(2)
    segmenter_actual = seg_name
    result = segmenter_cls(db, config).process_video(
        video_id, local_video, max_frames=args.max_frames
    )

```

#### 14. user (2026-07-07T14:46:30.115Z)

```

class CloudRunCompute(ComputeBackend):
    """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""

    def __init__(
        self,
        *,
        run_client: CloudRunJobClient,
        job_name: str,
        region: str,
        project: Optional[str] = None,
        storage_config: Optional[dict] = None,
        policy: ExecutionPolicy = DEFAULT_POLICY,
        token: Optional[str] = None,
        container_overrides: Optional[tuple[str, ...]] = None,
    ) -> None:
        self._client = run_client
        self._job_name = job_name
        self._region = region
        self._project = project
        self._storage_config = storage_config or {}
        self._policy = policy
        # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
        collide.
        # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic (tests).
        self._token = token
        self._container_overrides = (
            _DEFAULT_CONTAINER_OVERRIDES
            if container_overrides is None
            else tuple(container_overrides)
        )

```

```

def run_infer(
    self,
    spec: ComputeJobSpec,
    *,
    on_wait: "Callable[[float], None] | None" = None,
) -> ComputeResult:
    out_root = spec.out_uri.rstrip("/")
    token = self._token or uuid.uuid4().hex
    done_uri = f"{out_root}/_dispatch/{token}.done"
    argv: List[str] = [
        "infer",
        "--video-uri",
        spec.video_uri,
        "--out-uri",
        spec.out_uri,
        "--done-uri",
        done_uri,
    ]
    if spec.segmenter:
        argv += ["--segmenter", spec.segmenter]
    if spec.skip_validation:
        # The control-plane already validated + only dispatches on PASS ? tell the worker
not to
        # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
        argv.append("--skip-validation")
    for kv in (*spec.config_overrides, *self._container_overrides):
        argv += ["--set", kv]

    execution = self._client.run_job(
        self._job_name, argv, region=self._region, project=self._project
    )
    LOG.info(
        "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
        self._job_name,
        execution,
        done_uri,
    )

    try:
        marker = poll_until(
            lambda: self._read_marker(done_uri),
            self._policy,
            label="cloud-run-infer",
            logger=LOG,
            # Live heartbeat (#482): each still-waiting tick reaches the caller so
            # job.progress moves during the whole GPU run instead of freezing.
            on_tick=on_wait,
        )
    except PolicyTimeout as timeout_exc:
        marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
    video_id = str(marker.get("video_id", ""))
    # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-unreadable
/
    # garbled marker must not masquerade as rc=0. (The worker always writes both fields.)
    rc = int(marker.get("returncode", 1))
    if not video_id:
        LOG.warning(
            "cloud-run: job=%s completion marker present but empty/unreadable (no "
            "video_id) ? treating as failure",
            self._job_name,
        )
    rc = rc or 1
    outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
    report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}

```

```

LOG.info(
    "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
)
return ComputeResult(
    returncode=rc,
    outputs_uri=outputs_uri,
    video_id=video_id,
    report=report,
    execution=str(execution or ""),
)

def _handle_poll_timeout(
    self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
) -> dict:
    """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.

    The poll budget (`max_wait_sec`, default 1800s) can undercut the deployed job's own
    `--task-timeout` (3600s), so a slow-but-healthy worker may finish just past the
deadline:
    ONE final marker check rescues that completed GPU run instead of wasting it. If the
marker
    is genuinely absent, the execution is still running ? best-effort cancel it (otherwise
the

```

#### 15. user (2026-07-07T14:46:37.296Z)

```

backend/compute/cloud_run.py:17:The dispatched container is forced to run **INLINE**
(`compute_target=local_gpu` + native detector)
backend/compute/cloud_run.py:47:# (compute_target=local_gpu disarms the cmd_infer dispatch
guard; native = the L4 WASB runner.)
backend/compute/cloud_run.py:48:_DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
backend/compute/cloud_run.py:49:     "deployment.compute_target=local_gpu",
backend/compute/cloud_run.py:174:     _DEFAULT_CONTAINER_OVERRIDES

```

#### 16. user (2026-07-07T14:47:04.310Z)

```

)
segmentation_ran = True
if isinstance(result, Failure):
    print(f"[worker] segmenter FAILED: {result.message}")
    return _fail(3)
segments = result.value if hasattr(result, "value") else result
status = "success"
# Loud, never-silent diagnostic: a 0-segment success is almost always a misconfig, not
an
# empty match ? the cold-start failure mode (the substrate detector yielded no usable
# trajectories). Explain it so a run is analysable from the log alone (re-run -v for
more).
if not segments:
    detector_impl = (
        os.environ.get("RALLY_DETECTOR_IMPL")
        or getattr(config.indexing, "detector_impl", None)
        or "auto"
    )
    logger.warning(
        "infer produced 0 segments for video_id=%s (segmenter=%s, detector_impl=%s). "
        "The detector substrate yielded no usable trajectories/windows ? check the
detector "
        "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
        "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the key or "
        "indexing.skip_ai_handoff=true; cloud-serving now fails this at startup), native
"
        "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env produced no "
        "detections, or the input resolution differs from the tuned proxy resolution. "
        "Re-run with -v for the native command + frame counts.",

```

```

        video_id,
        segmenter_actual,
        detector_impl,
    )
finally:
    # Best-effort per-video observability report (never raises) ? parity with the main CLI.
    emit_indexer_report(
        db=db,
        video_id=video_id,
        video_path=local_video,
        config=config,
        val_results=val_results,
        val_context=val_context,
        segmenter_requested=args.segmenter,
        segmenter_actual=segmenter_actual,
        was_reingest=False,
        segmentation_ran=segmentation_ran,
    )

# 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
pushed = _serialize_and_push_outputs(
    scratch,
    args.out_uri,
    video_id,
    segments,
    status,
    storage_cfg,
    str(getattr(args, "video_uri", "") or ""),
)

print(
    f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushes)} artifact(s):"
)
for u in pushed:
    print("    ", u)

# M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so its
bucket-poll
# terminates (carries video_id ? the dispatcher's output path + rc=0). Worker FAILURES (rc
2/3)
# write their own marker via _fail() above, so the dispatcher fails FAST either way (no
30-min hang
# on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
if getattr(args, "done_uri", None):
    _write_done_marker(
        args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
    )
return 0

#: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter so a
#: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem collision).
_VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")

#: Leading ISO-date prefix on a canonical label name (`labels/<YYYY-MM-DD>_<name>.rallies.csv`,
#: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date but is NOT
#: part of the clip's join key, so it's stripped when deriving the videos/ stem.
_LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")

def _label_clip_name(fname: str) -> str:
    """Derive the clip name (the `videos/<name>.<ext>` join key) from a label filename.

    Strips both the `.rallies.csv` / `.csv` suffix and an optional leading ISO-date prefix,

```



so a

canonical dated label and a legacy bare label resolve to the SAME bare clip stem:

```
* ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094`` (canonical, date-stamped)
* ``GX020094.rallies.csv`` -> ``GX020094`` (legacy bare ? same stem)
* ``GX020094_proxy.rallies.csv`` -> ``GX020094_proxy`` (``_proxy`` is part of the name,
kept)
```

### 17. user (2026-07-07T14:47:24.632Z)

```
backend/api/job_worker.py:137:      # v8 compute-picker: the worker runs _execute_processing ?
_backend_dispatch_remote, which
backend/api/job_worker.py:364:      # Same normalization the execution path applies
(_maybe_dispatch_remote lowercases
backend/compute/base.py:58:      """Backend-agnostic compute. ``run_infer`` dispatches ONE
inference job and returns only once
backend/compute/cloud_run.py:116:      op = client.run_job(
backend/compute/cloud_run.py:171:      # None ? a fresh uuid4 per run_infer call (prod);
injected ? deterministic (tests).
backend/compute/cloud_run.py:189:      "infer",
backend/compute/cloud_run.py:190:      "--video-uri",
backend/compute/cloud_run.py:206:      execution = self._client.run_job(
backend/main.py:241:      _maybe_dispatch_remote,
backend/main.py:268:      # default (deployment.compute_target). Only honoured for these two
values; see _maybe_dispatch_remote.
backend/orchestration.py:4:Cloud Run dispatch + ingest path (``_maybe_dispatch_remote`` /
``_ingest_remote_segments``),
backend/orchestration.py:391:def _maybe_dispatch_remote(
backend/orchestration.py:589:      result = compute.run_infer(
backend/orchestration.py:728:      # _maybe_dispatch_remote when it dispatches. None ? nothing ran
(validation/config fail).
backend/orchestration.py:805:      remote = _maybe_dispatch_remote(
backend/worker/__main__.py:174:def _dispatch_remote(compute: Any, args: argparse.Namespace) ->
int:
backend/worker/__main__.py:192:      result = compute.run_infer(spec)
backend/worker/__main__.py:246:      return _dispatch_remote(compute, args)
backend/worker/__main__.py:270:      # --skip-validation (set by the dispatcher in
orchestration._maybe_dispatch_remote) skips it; the
backend/worker/__main__.py:710:      pi = sub.add_parser("infer", help="run inference on one video
URI and push outputs")
backend/worker/__main__.py:712:      "--video-uri", required=True, help="local path / local://
/ s3:// / gcs://"
```

### 18. user (2026-07-07T14:47:26.251Z)

```
backend/compute/base.py:37:      skip_validation: bool = False
backend/compute/cloud_run.py:199:      if spec.skip_validation:
backend/compute/cloud_run.py:202:      argv.append("--skip-validation")
backend/config/models.py:28:      min_blur_threshold: float = 20.0
backend/main.py:1513:      val_results, val_context = registry.run_all_with_context(
backend/orchestration.py:575:      # REJECT a clip this gate already passed (observed 2026-07-07:
passed at min_blur_threshold=8.0,
backend/orchestration.py:582:      skip_validation=True,
backend/orchestration.py:754:      val_results, val_context =
registry.run_all_with_context(local_video, config)
backend/pipeline/validators/base.py:50:      def run_all_with_context(self, video_path: str,
config: Any):
backend/pipeline/validators/base.py:78:      results, _ =
self.run_all_with_context(video_path, config)
backend/pipeline/validators/blur.py:72:      min_blur_threshold = val_cfg.min_blur_threshold
backend/pipeline/validators/blur.py:74:      if avg_variance < min_blur_threshold:
backend/pipeline/validators/blur.py:78:      message=f"Video quality rejected: Focus
check failed. Average sharpness score of {round(avg_variance, 1)} is below the minimum required
threshold of {min_blur_threshold}.",
backend/worker/__main__.py:189:      skip_validation=bool(getattr(args, "skip_validation",
```

```
False)),
backend/worker/__main__.py:268:    # (sharpness 11.9) PASSED the control-plane at
validation.min_blur_threshold=8.0, then the worker
backend/worker/__main__.py:270:    # --skip-validation (set by the dispatcher in
orchestration._maybe_dispatch_remote) skips it; the
backend/worker/__main__.py:272:    if getattr(args, "skip_validation", False):
backend/worker/__main__.py:274:        "[worker] validation SKIPPED (--skip-validation): the
dispatcher is the authoritative "
backend/worker/__main__.py:279:        val_results, val_context =
validator_registry.run_all_with_context(
backend/worker/__main__.py:744:        "--skip-validation",
```

## 19. user (2026-07-07T14:47:38.051Z)

```
        f"[JOBS] Recovered stale jobs from a previous run ? {requeued} compile re-queued, "
        f"{failed} process failed."
    )
    # P3 (#471): md5-keyed RUNNING remote jobs resume on outputs/<md5>/report.json ? never re-
dispatched.
    resume_interrupted_remote_jobs(global_config, db_instance)

    # 01/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-call
    # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a clip on
    # Google's servers forever). Best-effort, video-only, grace-gated; never blocks startup.
    _maybe_sweep_gemini_orphans(global_config)

    # CLI processing mode (no web UI needed). --app/--server always run the server, never CLI
single-shot.
    if args.video_path and not args.server and not args.app:
        print(f"[CLI] Processing video file: {args.video_path}")
    # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is fetched to
scratch.
    # The local-existence fast-fail only applies to a local input (stat the RESOLVED key,
not the
    # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error, not a
crash.
    storage_cfg = getattr(global_config, "storage", None)
    try:
        input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
    except ValueError as e:
        print(f"[FAIL] {e}")
        return
    if input_ref.backend == "local" and not os.path.exists(input_ref.key):
        print(f"[FAIL] Video file not found: {args.video_path}")
        return
    local_video = storage.acquire_local_input(args.video_path, storage_cfg)
    try:
        video_id = compute_video_id(local_video)
        was_reingest = db_instance.get_video(video_id) is not None

        # Validate (with-context variant surfaces ffprobe_meta for the report)
        print("[CLI] Validating...")
        val_results, val_context = registry.run_all_with_context(
            local_video, global_config
        )
        failures = [r for r in val_results if not r.passed]

        # Save video status
        status = "failed" if failures else "processed"
        db_instance.add_video(
            video_id, args.video_path, status, [r.model_dump() for r in val_results]
        )

    seg_name = args.segmenter or getattr(
        getattr(global_config, "indexing", None), "default_segmenter", "motion"
```

```

)
segmenter_actual = None
segmentation_ran = False
try:
    print("\n" + "=" * 40)
    print("VALIDATION SUMMARY")
    print("=" * 40)
    for r in val_results:
        status_symbol = "[PASS]" if r.passed else "[FAIL]"
        print(f"{status_symbol} {r.validator_name}: {r.message}")
    print("=" * 40 + "\n")

    if failures:
        print("[FAIL] Video failed checks. Indexing halted.")
        return

```

## 20. user (2026-07-07T14:47:39.333Z)

```

backend/main.py:241:     _maybe_dispatch_remote,
backend/main.py:268:     # default (deployment.compute_target). Only honoured for these two
values; see _maybe_dispatch_remote.

```

## 21. user (2026-07-07T14:48:03.982Z)

```

backend/compute/base.py:35:     # the gate already passed. Default False keeps the direct
``worker infer`` CLI (no control-plane)
backend/compute/cloud_run.py:5: 1. **Dispatch** a Cloud Run Job execution whose container runs
``python -m backend.worker infer
backend/compute/cloud_run.py:69:     """Start a Cloud Run Job execution running ``python -m
backend.worker <argv>``; return an
backend/orchestration.py:803:     # byte-identically. (player_pool/max_frames aren't threaded
to the worker ``infer`` CLI yet, so
backend/pipeline/detectors/native_wasb_runner.py:6:Linux GPU box (GCP / a SaaS worker): it runs
the **same** verified ``wasb_infer.py`` core
backend/pipeline/segmenters/trajectory_hybrid.py:187:     # caller (worker.cmd_infer,
main.py, orchestration) branches on isinstance(result, Failure).
backend/worker/__init__.py:3:``python -m backend.worker {infer|train}`` pulls inputs from a
storage URI (local / S3 /
backend/worker/__main__.py:3:     python -m backend.worker infer --video-uri <uri> --out-uri
<prefix> \
backend/worker/__main__.py:5:     python -m backend.worker train ...           # Gen-0 train-from-
bucket (follow-up)
backend/worker/__main__.py:160:     torch, and pre-M5 the worker's infer/train path was torch-
free (detection is delegated to a
backend/worker/__main__.py:271:     # direct ``worker infer`` CLI has no control-plane, so it
still validates by default.
backend/worker/__main__.py:396:     f"[worker] infer done: {len(segments)} segment(s); pushed
{len(pushed)} artifact(s):"
backend/worker/__main__.py:699:     prog="python -m backend.worker",
backend/worker/__main__.py:749:     "``worker infer`` CLI still validates.",

```

## 22. assistant (2026-07-07T14:48:51.276Z)

Let me verify the report emission path handles empty val\_context, since the new skip test mocks it out.

## 23. user (2026-07-07T14:48:53.765Z)

```

backend/main.py:1511:     # Validate (with-context variant surfaces ffprobe_meta for the
report)
backend/main.py:1513:     val_results, val_context = registry.run_all_with_context(
backend/main.py:1579:         val_context=val_context,

```

## 24. user (2026-07-07T14:48:59.302Z)

backend/reporting/\_\_init\_\_.py:29: def emit\_indexer\_report(

## 25. user (2026-07-07T14:49:03.917Z)

```
def emit_indexer_report(
    *,
    db,
    video_id: str,
    video_path: Optional[str],
    config: Optional[Dict[str, Any]],
    val_results: List[Any],
    val_context: Optional[Dict[str, Any]] = None,
    segmenter_requested: Optional[str] = None,
    segmenter_actual: Optional[str] = None,
    was_reingest: bool = False,
    segmentation_ran: bool = True,
) -> Optional[Dict[str, str]]:
    """Harvest + write the report. Returns the written file paths, or None if
    reporting is unavailable/disabled/failed. Never raises."""
    config = config or {}
    # Accept either a plain dict or a typed config object (e.g. backend.config.AppConfig).
    # Normalize to a plain nested dict so harvest's dict access works uniformly.
    if hasattr(config, "model_dump"):
        try:
            config = config.model_dump(mode="json")
        except Exception:
            config = {}
    # Consume run signals unconditionally (even when reporting is off) so the
    # process-local store never accumulates entries for finished runs.
    signals = run_signals.pop(video_id)
    if not OBSREPORT_AVAILABLE or not report_enabled(config):
        return None
    try:
        play_segments = db.query_play_segments(video_id, {}) if db is not None else []
        candidates = db.query_candidate_segments(video_id) if db is not None else []
        rec = harvest.record_run(
            video_id=video_id,
            video_path=video_path,
            config=config,
            val_results=val_results,
            val_context=val_context,
            play_segments=play_segments,
            candidates=candidates,
            segmenter_requested=segmenter_requested,
            segmenter_actual=segmenter_actual,
            was_reingest=was_reingest,
            segmentation_ran=segmentation_ran,
            run_signals=signals,
        )
        output_dir = (config.get("report") or {}).get("output_dir")
        res = rec.write(output_dir=output_dir)
        return res.paths or None
    except Exception: # pragma: no cover - defensive; reporting must never break a run
        return None
```

## 26. assistant (2026-07-07T14:49:25.808Z)

I've completed a thorough completeness-of-wiring review. Here is what I verified.

```
**Every `ComputeJobSpec` construction that dispatches a worker infer job:**
- `backend/orchestration.py:577` (`_maybe_dispatch_remote`) ? sets `skip_validation=True`.
Verified this is the only call site of `_maybe_dispatch_remote` (line 805), and it is reached
only after `registry.run_all_with_context` runs and any failure returns a `"failed"` response
before dispatch (lines 754-772). So the unconditional `skip_validation=True` is safe ? the
control-plane always validated with a PASS first. The cache-hit branch returns earlier and never
```

dispatches.

- ``backend/worker/__main__.py:184` (`_dispatch_remote`) ? forwards `skip_validation=bool(getattr(args, "skip_validation", False))` from the CLI. Correct: the direct-CLI dispatch path does NOT validate before dispatching (validation happens in the dispatched container), so defaulting to False means the container validates exactly once ? the intended behavior.`

**\*\*Every backend that turns a spec into worker argv:\*\*** ``CloudRunCompute`` is the only concrete ``ComputeBackend`` subclass, and ``get_compute_backend`` returns only ``CloudRunCompute`` or ``None``. It emits ``--skip-validation`` when ``spec.skip_validation`` is set (``cloud_run.py:199-202``). No second backend exists that would need to emit the flag.

**\*\*Other potential dispatch paths ? all confirmed to NOT dispatch a worker infer job (so no wiring needed):\*\***

- ``backend/main.py`` single-shot CLI (``args.video_path``) validates and runs the segmenter in-process; it imports ``_maybe_dispatch_remote`` but never calls it (only an import at line 241 + a comment at 268).
- ``resume_interrupted_remote_jobs`` resumes from existing ``outputs/<md5>/report.json`` and never re-dispatches.
- No subprocess/shell invocations of ``worker infer`` bypass ``ComputeJobSpec``.

**\*\*Consumer of the skipped-path state:\*\*** on the skip branch the worker sets ``val_results=[]``, ``val_context={}``. The one downstream consumer, ``emit_indexer_report``, wraps everything in a catch-all ``try/except`` and is documented never-raises (``backend/reporting/__init__.py``), and on the cloud path the control-plane emits its own authoritative report from its real ``val_results``. So the empty context cannot crash the job or corrupt the ingested result.

No wiring gaps found: every dispatch path carries the flag when it should, and the single control-plane dispatch site is guaranteed to have validated (PASS) before setting ``skip_validation=True``.

## 27. user (2026-07-07T14:49:25.816Z)

Structured output provided successfully